

Implementation Prompt — "Orchestra" Multi-Agent Orchestrator (0 dan)

Bu promptni to'liq nusxalab Claude Code'ga (yoki boshqa AI agentga) bering.
U butun loyihani noldan quradi. Prompt ichidagi talablar majburiy (MUST).

ROLE

Sen tajribali Python backend muhandisidan. Vazifa — Claude Agent SDK asosida ishlovchi, ishonchli, qayta tiklanadigan (resumable) **multi-agent orkestratsiya tizimini** noldan qurish. Kod toza, type-hinted, async va testlar bilan bo'lsin.

MAQSAD

Bitta vazifa berilganda tizim avtomatik ravishda:

1. **Planner** agent vazifani tahlil qiladi, strategiya yozadi va kichik vazifalarga (subtask) bo'ladi (JSON ko'rinishida).
2. Orchestrator har bir mustaqil subtask uchun alohida **Executor** agentni **parallel** ishga tushiradi.
3. Har bir bajarilgan ish **Reviewer/Tester** agentga uzatiladi — u to'liq testdan o'tkazadi va tasdiqlaydi. Test o'tmasa — qayta ijroga qaytadi.
4. Har bir holat o'zgarishida **Telegram** bot orqali hisobot yuboriladi (agent roli + agent ID + task ID + status).
5. Asosiy orchestrator LLM kontekstini to'ldirmaydi — u bazadan o'qiydigan yengil dispatcher. Shuning uchun u "doim bo'sh" turadi va hech narsani unutmaydi: barcha holat ma'lumotlar bazasida saqlanadi.

ASOSIY TAMOYIL (eng muhim)

- Tizimning yagona haqiqat manbasi — ma'lumotlar bazasi, LLM emas.
 - Orchestrator har sikl bazadan tugamagan vazifalarni o'qiydi → process qulasa yoki server qayta yuklansa, qoldiqdan davom etadi.
 - Har bir agent chaqiruvi **toza, izolyatsiyalangan sessiya** — faqat o'z subtask'i bilan.
-

TEKNOLOGIK STEK (MUST)

- Python 3.11+

- `claude-agent-sdk` (asosiy)
 - `aiosqlite` (default baza) — interfeys orqali Postgres'ga ham almashtirilsin
 - `httpx` (Telegram HTTP)
 - `pydantic` (config va structured output validatsiya)
 - `pytest` + `pytest-asyncio` (testlar)
 - Docker + docker-compose
 - `uv` yoki `pip` (paket boshqaruvi)
-

LOYIHA TUZILMASI (MUST — aynan shu tuzilmani yarat)

```
orchestra/
├─ pyproject.toml
├─ .env.example
├─ README.md
├─ Dockerfile
├─ docker-compose.yml
├─ src/orchestra/
│   ├─ __init__.py
│   ├─ config.py           # pydantic Settings (.env'dan o'qiydi)
│   ├─ models.py          # Task, AgentRun, TaskStatus(Enum) dataclasses
│   ├─ db.py              # AsyncDB interfeys + SQLiteDB implementatsiya
│   └─ agents.py          # AgentDefinition registry
(planner/executor/reviewer)
├─ runner.py              # run_agent(): SDK query() o'rami
├─ reporter.py            # TelegramReporter
├─ orchestrator.py        # main loop + handle_task + parallel ijro
└─ telegram_bot.py        # inbound: /task buyrug'i orqali vazifa qabul
qiladi
├─ guards.py              # xavfli Bash komandalarni tekshiruvchi
├─ cli.py                 # entrypoint: submit / run
├─ .claude/
│   └─ settings.json      # Claude Code hook'lari (ixtiyoriy path uchun)
├─ scripts/
│   └─ guard_bash.py      # PreToolUse hook skripti
└─ tests/
    ├─ test_db.py
    ├─ test_runner.py
    └─ test_orchestrator.py
```

MA'LUMOTLAR MODELI (MUST)

`TaskStatus` enum: `PENDING, ANALYZING, PLANNED, EXECUTING, REVIEWING, TESTING, DONE, FAILED`.

tasks jadvali:

- `id` TEXT PK (uuid4)
- `parent_id` TEXT NULL (root vazifa uchun NULL)
- `kind` TEXT ('root' | 'subtask')
- `title` TEXT
- `description` TEXT
- `strategy` TEXT NULL (planner natijasi)
- `deps` TEXT NULL (JSON: bog'liq subtask id'lari)
- `status` TEXT (TaskStatus)
- `result` TEXT NULL
- `attempts` INTEGER DEFAULT 0
- `agent_id` TEXT NULL
- `session_id` TEXT NULL (SDK sessiyasi, resume uchun)
- `created_at`, `updated_at` TIMESTAMP

agent_runs jadvali:

- `id` TEXT PK (masalan "executor-ab12cd")
- `task_id` TEXT FK
- `role` TEXT ('planner'|'executor'|'reviewer')
- `model` TEXT
- `status` TEXT
- `started_at`, `finished_at` TIMESTAMP

events jadvali (audit + TG log):

- `id` INTEGER PK AUTOINCREMENT
- `task_id` TEXT, `agent_id` TEXT, `status` TEXT, `message` TEXT
- `created_at` TIMESTAMP

`db.py` da `AsyncDB` abstrakt interfeysini yoz (`next_pending_root`, `save_task`, `save_subtasks`, `update_status`, `set_result`, `list_subtasks`, `increment_attempts`, `log_event`). `SQLiteDB` — aiosqlite implementatsiyasi. Migratsiya/initdb funksiyasi jadvallarni avtomatik yaratsin.

AGENTLAR (MUST — `agents.py`)

`claude_agent_sdk.AgentDefinition` orqali uchta agent:

1. **planner** (model: `opus`, tools: `Read, Grep, Glob`)
 - System prompt: vazifani tahlil qiladi, strategiya yozadi, subtask'larga bo'ladi. FAQAT quyidagi JSON qaytaradi (boshqa hech narsa, markdown ham yo'q):

json

```
{"strategy": "...", "subtasks": [{"id": "s1", "title": "...", "deps": []}]}
```

2. **executor** (model: `sonnet`, tools: `Read, Write, Edit, Bash, Grep, Glob`)
 - Berilgan bitta subtask'ni oxirigacha bajaradi, natija/o'zgarishlar xulosasini qaytaradi.
3. **reviewer** (model: `sonnet`, tools: `Read, Bash, Grep, Glob`)
 - Bajarilgan ishni ko'rib chiqadi, testlarni ishga tushiradi. FAQAT JSON:

json

```
{"passed": true, "report": "..."} 
```

RUNNER (MUST — `runner.py`)

`async def run_agent(role, prompt, *, session_id=None) -> tuple[str, str]:`

- `claude_agent_sdk.query()` chaqiradi.
 - `ClaudeAgentOptions`: `allowed_tools` ga "Agent" ni qo'sh, `permission_mode="bypassPermissions"`, `agents={role: AGENTS[role]}`, `resume=session_id` (agar berilgan bo'lsa).
 - In-process **hooks** (`options.hooks`) orqali `PreToolUse` da xavfli Bash komandalarni `guards.py` bilan bloklaydi; `SubagentStop` / `PostToolUse` da reporterга signal yuboradi.
 - Streamdan yakuniy `result` matnini va `session_id` ni ajratib qaytaradi.
 - JSON kutilganda toza JSON parse qiladi (``json fence'larni tozalab).
-

ORCHESTRATOR (MUST — `orchestrator.py`)

```
async def handle_task(task):
```

1. status → ANALYZING, TG hisobot ("planner 🟡 tahlil").
2. `run_agent("planner", task.description)` → JSON parse → strategy + subtasks.
3. subtasks'ni bazaga yoz, status → PLANNED, TG ("🟢 N ta subtask").
4. Bog'liqliksiz (`deps == []`) subtask'larni `asyncio.gather` bilan PARALLEL ishga tushir. Bog'liqlari `deps` tugagach navbat bilan.
5. Har subtask uchun:
 - EXECUTING, TG → `run_agent("executor", subtask.title)` → result saqla.
 - REVIEWING, TG → `run_agent("reviewer", review_prompt)` → JSON.
 - `passed=True` → DONE; aks holda `attempts+1`, agar `attempts < MAX_RETRY` bo'lsa EXECUTING'ga qaytar (retry), aks holda FAILED. Har holatda TG.
6. Barcha subtask DONE bo'lsa, root task DONE, yakuniy TG xulosa.

```
async def orchestrator_loop():
```

- Cheksiz sikl: `db.next_pending_root()` → bor bo'lsa `handle_task`, yo'q bo'lsa `asyncio.sleep(POLL_INTERVAL)`. Ishga tushganda tugamagan vazifalarni bazadan tiklab davom ettiradi.

`MAX_RETRY` va `POLL_INTERVAL` config'dan olinsin.

TELEGRAM (MUST)

```
reporter.py — TelegramReporter.report(agent_id, role, task_id, status_text):
```

- `httpx` bilan `sendMessage` POST. Format (Markdown):

```
*{role}* `{agent_id}`  
📋 task `{task_id[:8]}`  
{status_text}
```

- Har report `events` jadvaliga ham yoziladi.

```
telegram_bot.py (inbound, ixtiyoriy lekin yoz):
```

- Long-polling (`getUpdates`) yoki webhook orqali `/task <matn>` buyrug'ini qabul qilib, bazaga root task qo'shadi (PENDING). Boshqa buyruqlar: `/status <id>`, `/tasks`

(oxirgi 10 ta).

XAVFSIZLIK (MUST — `guards.py` + `.claude/settings.json`)

- `guards.py`: `is_dangerous(cmd: str) -> bool` — `rm -rf`, `git push`, `{() }`, `mkfs`, `dd if=`, `curl ... | sh`, `shutdown`, `reboot` kabilarni qora ro'yxat orqali bloklaydi. Konfiguratsiyalanadigan bo'lsin.
 - Runner'dagi `PreToolUse` hook shu funktsiyani chaqiradi; xavfli bo'lsa toolni rad etadi (deny) va TG'ga ogohlantirish yuboradi.
 - Executor — `Bash` bor; Reviewer — `Bash` bor lekin `Write/Edit` YO'Q.
 - `.claude/settings.json` — Claude Code orqali ishlatilsa, `PreToolUse` Bash matcher `scripts/guard_bash.py` ni chaqirsin (exit 2 = bloklash).
 - README'da: production'da executor'larni alohida cheklangan Linux foydalanuvchi yoki konteyner ostida ishga tushirish tavsiya etilsin.
-

CONFIG (MUST — `config.py`)

pydantic `Settings`: `ANTHROPIC_API_KEY`, `TG_BOT_TOKEN`, `TG_CHAT_ID`, `DB_PATH` (default `orchestra.db`), `POLL_INTERVAL=2`, `MAX_RETRY=2`, `PLANNER_MODEL=opus`, `EXECUTOR_MODEL=sonnet`, `REVIEWER_MODEL=sonnet`, `MAX_PARALLEL=5`. `.env.example` faylini ham yarat.

CLI (MUST — `cli.py`)

- `python -m orchestra.cli run` — orchestrator loop'ini ishga tushiradi.
 - `python -m orchestra.cli submit "<vazifa matni>"` — bazaga root task qo'shadi.
 - `python -m orchestra.cli bot` — Telegram inbound bot'ini ishga tushiradi.
 - `python -m orchestra.cli status <id>` — vazifa holatini ko'rsatadi.
-

DOCKER (MUST)

- `Dockerfile`: `python:3.11-slim`, paketlar, `CMD ["python", "-m", "orchestra.cli", "run"]`.
- `docker-compose.yml`: `orchestrator` va `bot` servislari, umumiy volume (baza fayli), `.env` orqali muhit o'zgaruvchilari.

TESTLAR (MUST — `tests/`)

- `test_db.py`: jadvallar yaratiladi, task qo'shiladi/yangilanadi, next_pending to'g'ri ishlaydi.
 - `test_runner.py`: `run_agent` mock'langan SDK bilan JSON parse qiladi; xavfli Bash bloklanadi.
 - `test_orchestrator.py`: planner mock (2 subtask qaytaradi) → 2 executor parallel chaqiriladi → reviewer pass → task DONE. Retry stsenariysi ham test qilinsin.
 - SDK chaqiruvlari mock orqali (haqiqiy API'siz) testlanadi.
-

QABUL MEZONLARI (Acceptance — MUST)

1. `submit` → `run` qilinganda Telegram'da ketma-ket xabarlar keladi: planner, har bir executor (ID bilan), har bir reviewer (ID bilan), yakuniy xulosa.
 2. Subtask'lar haqiqatan parallel ishlaydi (log'da vaqtlar ustma-ust tushadi).
 3. Reviewer fail qaytarsa, subtask qayta ijro etiladi (MAX_RETRY gacha).
 4. Orchestrator to'xtatilib qayta ishga tushirilsa, tugamagan vazifa davom etadi (baza saqlangani uchun).
 5. Xavfli Bash komandasi bloklanadi va TG'ga ogohlantirish boradi.
 6. `pytest` to'liq yashil.
 7. `README.md` to'liq: o'rnatish, .env, ishga tushirish, arxitektura, xavfsizlik.
-

ISH TARTIBI (sen, AI agent, quyidagicha ishla)

1. Avval `pyproject.toml`, `config.py`, `models.py`, `db.py` ni yoz va `test_db.py` ni o'tkaz.
2. So'ng `agents.py`, `runner.py`, `guards.py` + `test_runner.py`.
3. So'ng `reporter.py`, `orchestrator.py` + `test_orchestrator.py`.
4. So'ng `telegram_bot.py`, `cli.py`.
5. Oxirida Docker, `.claude/settings.json`, `scripts/guard_bash.py`, `README.md`.
6. Har bosqichda `pytest` ishga tushirib, yashilga keltir, keyin davom et.
7. Tugagach, `README.md` da qisqa "qanday ishlaydi" diagrammasini matn ko'rinishida ber va misol ishga tushirish buyrug'ini ko'rsat.

Boshla.

